Universidad Tecnológica Nacional

Facultad Regional Córdoba

Ingeniería en Sistemas de Información

Ingeniería y Calidad de Software

TEMA: Desarrollo Conducido por Pruebas

Unidad Nro. 4

Trabajo Práctico Nro. 3

Año: 2026

Docentes:
- Ing. Judith Meles
- Ing. Valeria Abdala
- Salvador Barbera

Curso: 4K1
Grupo Nro: 6

Integrantes

Nombre y apellido	Legajo	Email
Juan Ignacio Arrigoni	407069	juani.arrigoni@gmail.com
Lautaro Francisco Castillo	400998	castillolautarof@gmail.com
Francisco Dalmazzo	400193	fgdalmazzo@gmail.com
Ezequiel Deza	401006	dezaezequiel5@gmail.com
Gina Eusebi	400202	ginaeusebi9@gmail.com
Juan Ignacio Fanfliet	400879	juanignaciofanfliet@gmail.com
Olivia García Osella	400032	oliviaosella@gmail.com
Delfina Guerci	401782	delfinaguerci@gmail.com
Ignacio Facundo Lescano Barreto	400523	facuiglescano14@gmail.com
Ignacio Martín	404677	ignamartin112@gmail.com
Elena Adriana Moyano Matlega	401342	moyanoelena7@gmail.com
Catalina Pérez	402462	cataaperezz04@gmail.com
Diego Matías Petitto	400258	dmpetitto2004@gmail.com
Ignacio Samocachan	400621	ignasamo2@gmail.com
Pedro Torcivia	400091	pedritorcivia@gmail.com

Índice

1. Introducción 4
2. Consigna y alcance 4
3. Solución implementada 5
4. Aplicación del enfoque TDD 6
5. Evidencia por subgrupo 7
5.1 Consulta de actividades y turnos 7
5.2 Términos y aceptación 8
5.3 Validación de participantes 9
5.4 Confirmación correo y QR 10
5.5 Reservas y cupos 11
6. Integración de los módulos 12
7. Pruebas automatizadas 13
8. Decisiones de diseño 14
9. Ejecución y configuración 14
10. Conclusión 15
11. Glosario 15

1. Introducción

El presente informe documenta la implementación de la User Story Inscribirme a actividad de EcoHarmony Park mediante Desarrollo Conducido por Pruebas. El trabajo se realizó en Python con FastAPI para el backend y React con TypeScript para la interfaz. La construcción del comportamiento de negocio se inició desde pruebas automatizadas y quedó registrada en ramas y commits que identifican las etapas Red, Green y Refactor.

El resultado integra consulta de actividades y turnos, aceptación de términos, validación de participantes, gestión de reservas y cupos, generación del comprobante QR y envío de la confirmación por correo. La suite integrada alcanza 171 casos automatizados y utiliza dependencias falsas o temporales para evitar envíos reales y aislar la persistencia durante las pruebas.

2. Consigna y alcance

La consigna solicitó implementar una funcionalidad mediante TDD y aplicar el ciclo Red Green Refactor para construir el código desde las pruebas. Por tratarse del Grupo 6, se seleccionó la User Story Inscribirme a actividad.

2.1 Objetivo y propósito

El objetivo fue diseñar y construir una solución funcional a partir de pruebas automatizadas, incorporando criterios de calidad desde el inicio. El propósito fue ejercitar la disciplina TDD y utilizar cada prueba como especificación ejecutable del comportamiento esperado.

2.2 Entradas y entregables

- Conceptos teóricos y bibliografía de la Unidad 4.
- Enunciado de EcoHarmony Park, User Stories del Trabajo Práctico 2 y reglas de estilo del grupo.
- Código funcional de la inscripción a actividades y pruebas automatizadas de sus casos.
- Evidencia del ciclo TDD y justificación de las decisiones de diseño e integración.

3. Solución implementada

La aplicación permite seleccionar una actividad, consultar fechas y turnos disponibles, cargar participantes, aceptar las condiciones vigentes y confirmar la reserva. El backend vuelve a validar las reglas en el registro definitivo, descuenta cupos de forma atómica, persiste la reserva y genera un QR legible. El correo se procesa después de confirmar la reserva; si falla, la inscripción conserva su validez y puede reintentarse sin descontar cupos nuevamente.

3.1 Arquitectura

Componente	Responsabilidad
Frontend React	Presenta el flujo y consume exclusivamente datos y reglas publicados por la API.
API FastAPI	Expone actividades, calendario, horarios, términos, validación y registro de inscripciones.
Servicios de dominio	Aplican reglas de participantes, aceptación, reservas, QR y notificaciones.
Persistencia SQLite	Conserva reservas y controla cupos por actividad, fecha y horario.
Servicios externos	SMTP entrega el correo; la librería qrcode genera el comprobante PNG.

3.2 Flujo funcional

1. Consultar el catálogo oficial de actividades.
2. Consultar el calendario y los turnos reservables con sus cupos.
3. Cargar los participantes y validar sus datos mediante el backend.
4. Obtener y aceptar la versión vigente de los términos de la actividad.
5. Registrar la inscripción; el servidor revalida apertura, cupos, participantes y términos.
6. Mostrar el QR y el estado del correo. Si el envío falla, permitir el reintento de la notificación.

3.3 Contratos principales

Método y ruta	Uso
GET /actividades	Catálogo, capacidad, edad mínima, talle y talles disponibles.
GET /actividades/{actividad}/calendario	Ventana, cierres, fechas, horarios, identificadores y cupos.
POST /inscripciones/validar	Validación informativa de participantes.
GET /actividades/{actividad}/terminos	Texto y versión de las condiciones vigentes.
POST /inscripciones	Confirmación integral y persistencia de la reserva.

4. Aplicación del enfoque TDD

El trabajo se dividió por responsabilidades para permitir ciclos pequeños. Cada subgrupo desarrolló una porción del comportamiento en una rama independiente y registró en los mensajes de commit la fase del ciclo aplicada. Luego se integraron los contratos y se agregaron pruebas que cubren la colaboración entre módulos.

4.1 Ciclo Red Green Refactor

Fase	Aplicación en el proyecto
Red	Se escribió una prueba que expresaba el comportamiento faltante y se verificó el fallo esperado.
Green	Se agregó la implementación mínima necesaria para que la prueba y la suite existente pasaran.
Refactor	Se reorganizó el diseño, se extrajeron helpers y se eliminaron duplicaciones sin cambiar el comportamiento.

4.2 Ramas de trabajo

Rama	Responsabilidad
tdd/consultar-actividad	Catálogo, turnos, cierres, anticipación y cupos visibles.
tdd/terminos-confirmacion	Términos por actividad y aceptación obligatoria.
tdd/cargar-participante	Cantidad, nombre, DNI, edad, talle y duplicados.
tdd/envio-confirmacion-inscripcion	Resumen, correo, QR e inyección de dependencias.
gestion-reservas-cupos	Reserva, persistencia, cupos, solapamiento y concurrencia.
tdd/integracion/integracion-grupos-1-a-5	Contrato HTTP y orquestación de los cinco módulos.
tdd/frontend	Interfaz conectada al contrato integrado del backend.

La evidencia incluye un revert explícito del commit que incorporaba SMTP sin una prueba previa. La funcionalidad se reimplementó después con su ciclo Red y Green, lo que muestra que el proceso TDD se utilizó también como criterio de control del historial.

5. Evidencia por subgrupo

5.1 Consulta de actividades y turnos

El primer módulo construyó el catálogo y la consulta de turnos. Las pruebas definieron los horarios entre las 09:00 y las 17:00, la independencia de cupos por turno, el estado sin cupos y los cierres de los lunes, el 1 de enero y el 25 de diciembre. También se agregó una ventana adoptada por el backend: el turno se muestra cuando faltan al menos 15 minutos y como máximo 24 horas.

Evidencia de commits

Fase	Commit	Evidencia
Red	7136bf3	Prueba que consulta turnos un lunes y espera una lista vacía.
Green	d0f0d7a	Implementación del cierre semanal.
Red	2101455	Prueba de turno completo sin disponibilidad.
Green	26cdd7d	Estado y motivo para el turno lleno.
Refactor	5dcebf4	Reorganización del cálculo independiente de cupos.
Red	51a82e1	Caso con menos de 15 minutos de anticipación.
Green	580a29f	Filtro por anticipación mínima.
Green	72e2669	Filtro para turnos a más de 24 horas.

Decisiones de diseño

- Representar cada turno con actividad, fecha, hora, cupos disponibles, disponibilidad y motivo.
- Consultar la ocupación mediante una dependencia para aislar el cálculo durante los tests.
- Centralizar los cierres y los límites de anticipación en el backend para que el frontend no replique reglas.

5.2 Términos y aceptación

El segundo módulo estableció que cada inscripción debe aceptar términos correspondientes a la misma actividad. Los casos parametrizados cubren las cuatro actividades, la ausencia de aceptación, la aceptación cruzada y la prioridad de errores. Los textos se separaron de la lógica para permitir que cambien sin alterar la validación.

Evidencia de commits

Fase	Commit	Evidencia
Red	2ffade1	Prueba de rechazo cuando los términos no fueron aceptados.
Green	65665f6	Excepción de términos no aceptados.
Red	feb0270	Prueba para aceptación perteneciente a otra actividad.
Green	65e00e4	Validación de correspondencia entre actividad y aceptación.
Refactor	64acfe9	Unificación de combinaciones de actividades en tests parametrizados.
Red	1d91ec3	Prueba inicial de obtención de términos con NotImplementedError.
Green	2a1d74b	Texto mínimo para satisfacer el contrato.
Refactor	e1d496b	Separación del texto ficticio y la función de consulta.

Decisiones de diseño

- Modelar la aceptación como un objeto con actividad, indicador de aceptación y versión.
- Validar la aceptación antes de modificar cupos o persistir una reserva.
- Enviar la versión mostrada al usuario y rechazar una confirmación si los términos cambiaron.

5.3 Validación de participantes

El tercer módulo definió las reglas de los visitantes: debe existir al menos un participante, la cantidad declarada debe coincidir con la lista, el nombre debe contener partes alfabéticas, el DNI debe ser numérico y positivo, la edad no puede ser negativa y debe respetar el mínimo de la actividad, y el talle debe pertenecer al catálogo cuando sea obligatorio. También se rechazan DNI repetidos dentro de la misma solicitud.

Evidencia de commits

Fase	Commit	Evidencia
Red	bc1f855	Prueba inicial de existencia de las validaciones.
Green	63bda28	Implementación mínima del contrato.
Red	8e1a4c6	Prueba de nombre válido.
Green	686763f	Validación del nombre del participante.
Red	0d804a1	Prueba de talle obligatorio.
Green	1254d75	Validación de talle según actividad.
Red	bccb5b4	Prueba de DNI repetidos.
Green	ebeb551	Rechazo de duplicados.
Refactor	dcd2e4f	Helper de construcción de participantes para reducir duplicación.

Decisiones de diseño

- Mantener una validación individual reutilizable y una validación del conjunto.
- Usar el enum Actividad como fuente de cupo, edad mínima y obligatoriedad de talle.
- No inventar límites de DNI o edad que no estén definidos por el contrato probado.

5.4 Confirmación correo y QR

El cuarto módulo prepara un resumen legible, genera un QR PNG y envía el comprobante por correo. El diseño utiliza inyección de dependencias: CorreoFalso y SMTPFalso permiten verificar destinatario, contenido, adjunto y orden de las operaciones sin conectarse a un servidor real. Las credenciales se leen desde variables de entorno y no forman parte del repositorio.

Evidencia de commits

Fase	Commit	Evidencia
Red	e336009	Prueba inicial del resumen de confirmación.
Green	d867288	Implementación mínima; no se refactorizó porque ya era suficiente.
Red	4dd9ab7	Prueba de envío mediante CorreoFalso.
Green	7a6217c	Invocación mínima del emisor inyectado.
Red	c69376c	Prueba que exige el QR dentro del correo.
Green	73f1b76	Integración de generar_qr en el servicio.
Refactor	c2e39ac	Reorganización de imports y mejora del resumen.
Control	027a242	Revert del código SMTP agregado sin test previo.

Decisiones de diseño

- Separar la generación de QR del envío de correo para probarlos en forma independiente.
- Usar smtplib y EmailMessage, con configuración SMTP externa mediante .env.
- Confirmar la reserva aunque el correo falle y registrar el estado para permitir reintentos.

5.5 Reservas y cupos

El quinto módulo administra la creación de reservas y la disponibilidad. Las pruebas comenzaron con capacidades iniciales, continuaron con descuento e independencia de cupos y cubrieron errores sin efectos colaterales, persistencia, DNI superpuesto y dos solicitudes concurrentes por los últimos lugares. La versión integrada reemplazó la persistencia en memoria inicial por SQLite sin perder el contrato probado.

Evidencia de commits

Fase	Commit	Evidencia
Red	b096643	Prueba de descuento de cupos en una reserva válida.
Green	01e7125	Descuento mínimo de la cantidad solicitada.
Refactor	bba8d41	Mejora de la estructura del control de cupos.
Red	3e0e37d	Prueba de rechazo sin persistencia ni modificación de cupos.
Green	d09cb7f	Validaciones previas a cualquier mutación.
Red	cb85c56	Dos solicitudes simultáneas compiten por los últimos lugares.
Green	3bceeb5	Exclusión mutua para impedir la sobreventa.
Refactor	7c41d5c	Separación de lecturas internas para evitar bloqueos.

Decisiones de diseño

- Ejecutar todas las validaciones antes de persistir o descontar cupos.
- Usar una operación atómica para comprobar y consumir disponibilidad.
- Separar cupos y DNI por actividad, fecha e identificador de turno.

6. Integración de los módulos

La integración requirió un contrato común para actividad, fecha, horario, participantes, aceptación, correo y respuesta. Los endpoints conservan las reglas de los subgrupos y las aplican nuevamente al confirmar. Esta decisión evita que una consulta previa de disponibilidad se confunda con una reserva efectiva.

6.1 Decisiones de integración

Problema	Resolución integrada
Modelos diferentes entre ramas	Se consolidaron enums y modelos Pydantic en schemas.py.
Cupos consultados antes de confirmar	El POST definitivo repite la validación y descuenta en una transacción.
Persistencia inicialmente en memoria	SQLite conserva reservas y evita sobreventa mediante transacciones.
QR corto en la rama inicial	El comprobante final contiene identificador y datos legibles de la reserva.
Fallo de SMTP	La reserva permanece confirmada; se informa correo_estado y se permite reintentar.
Términos cambiantes	La versión aceptada se compara con la vigente antes de registrar.

6.2 Frontend basado en el backend

La interfaz no conserva un catálogo paralelo de reglas. Obtiene del backend actividades, cupos, edades mínimas, talles, cierres, ventana de anticipación, horarios e identificadores. La validación de participantes se ejecuta mediante POST /inscripciones/validar y la confirmación vuelve a pasar por el backend. En el frontend permanecen únicamente el estado temporal del formulario y los recursos visuales asociados a cada actividad.

6.3 Commits de integración

Commit	Resultado
5140e41	Integración de actividades y participantes con términos, reservas y envío.
91d4d5f	Cierre de QR, SMTP, términos y persistencia.
3749398	Comprobante QR legible y datos organizados.
442064a	Backend integrado con el frontend.
c0ad04c	Backend convertido en fuente de reglas de la interfaz.

7. Pruebas automatizadas

La verificación final recoge 171 casos en 15 archivos. Los conteos incluyen parametrizaciones, por lo que superan la cantidad de funciones cuyo nombre comienza con test. La suite utiliza relojes controlados, bases temporales y servicios falsos para mantener resultados repetibles.

Archivo	Casos	Alcance
test_consulta_actividades.py	16	Catálogo, turnos, cierres, anticipación y cupos
test_endpoints.py	7	Contratos HTTP iniciales
test_terminos.py	12	Textos y estabilidad de términos
test_aceptacion.py	32	Aceptación y prioridad de errores
test_participantes.py	12	Nombre, DNI, edad, talle y cantidad
test_notificaciones.py	8	Resumen, destinatario e inyección
test_generacion_qr.py	3	PNG, contenido y unicidad
test_comprobante_legible.py	1	Coherencia del comprobante
test_correo_smtp.py	1	SMTP con servidor falso
test_reservas.py	12	Cupos, persistencia y concurrencia
test_integracion_inscripcion.py	3	Contrato compartido
test_integracion.py	8	Orquestación de servicios
test_integracion_completa.py	20	Recorrido HTTP y reglas cruzadas
test_api_endpoints.py	21	API, calendario, cierres y CORS
test_cierre_backend.py	15	SQLite, QR completo y reintentos

7.1 Resultado final

```
..................................................................... [ 42%]
..................................................................... [ 84%]
...........................                                          [100%]
171 passed
```

8. Decisiones de diseño

ID	Decisión	Justificación
D1	Reglas en el backend	La API es la única fuente de reglas; la interfaz solo representa su respuesta.
D2	Validación repetida	La consulta previa mejora la interacción, pero el POST definitivo protege la integridad.
D3	Persistencia SQLite	Permite recuperar reservas y realizar el descuento dentro de una transacción.
D4	Dependencias inyectables	Correo, reloj y persistencia se sustituyen en tests sin efectos externos.
D5	Reserva independiente del correo	Una caída de SMTP no invalida ni duplica la reserva.
D6	Contratos tipados	Pydantic valida solicitudes y respuestas; los errores HTTP expresan la causa.

9. Ejecución y configuración

9.1 Backend

```
cd 04_Trabajos_Practicos/Evaluables/TPE03/G6_COD_InscripcionActividad/backend
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload
```

9.2 Frontend

```
cd 04_Trabajos_Practicos/Evaluables/TPE03/G6_COD_InscripcionActividad/frontend
npm ci
npm run dev
```

9.3 Variables de correo

La configuración SMTP se guarda en backend/.env y se toma de backend/.env.example. SMTP_HOST, SMTP_USER y SMTP_PASSWORD son obligatorias. La contraseña debe ser una contraseña de aplicación y el archivo .env permanece excluido del control de versiones.

9.4 Ejecución de pruebas

```
cd backend
.\.venv\Scripts\python.exe -m pytest -q

cd ../frontend
npm run build
npm run lint
```

10. Conclusión

La User Story Inscribirme a actividad quedó implementada como un flujo completo y verificable. Los ciclos registrados muestran que las reglas se incorporaron a partir de pruebas pequeñas y que el diseño se ajustó sin perder comportamiento. La integración añadió pruebas de contrato y de recorrido para comprobar que las piezas desarrolladas por separado funcionaran juntas.

La solución final conserva las reglas en el backend, protege los cupos mediante persistencia transaccional, desacopla el envío de correo y expone al frontend un contrato único. La suite de 171 casos permite repetir la verificación y detectar regresiones en actividades, participantes, términos, reservas, comprobantes y endpoints.

11. Glosario

Término	Definición
TDD	Desarrollo Conducido por Pruebas; práctica en la que las pruebas guían la implementación.
Red	Etapa en la que una prueba nueva falla por la ausencia del comportamiento esperado.
Green	Etapa en la que se agrega el código mínimo para que la prueba pase.
Refactor	Mejora del diseño que conserva el comportamiento verificado por la suite.
Triangulación	Uso de casos adicionales para evitar una implementación demasiado específica.
Doble de prueba	Objeto controlado que sustituye una dependencia real, como CorreoFalso o SMTPFalso.
Prueba de integración	Caso que verifica la colaboración entre módulos o capas.
Regresión	Fallo que reaparece luego de un cambio y que una prueba automatizada permite detectar.
Atomicidad	Propiedad por la cual una operación se completa en su totalidad o no modifica el estado.
Contrato HTTP	Estructura acordada para solicitudes, respuestas, estados y errores de una API.

11.1 Repositorio

https://github.com/oliviaosella/G6_Ingenieria_Calidad_de_Software_4K1_2026.git